DAILY FUSION CODE · WEEK 1

Heating a plasma: Euler's method, second try

Python | Sun 18 Oct 2026 | NUM-4.1 | Level 1 | ~20 min

A retry of Euler's method with a new problem. When the heating switches on, the energy W stored in a tokamak plasma rises until the heating power P balances the losses W/τ_E:

$$\frac{dW}{dt} = P - \frac{W}{\tau_E}, \qquad W(0) = 0$$

Take $P = 10$ MW and $\tau_E = 2$ s, so W is in MJ. The starter file has the whole program; you fill in five short `TODO` gaps and see how Euler's error shrinks with Δt.

New to Euler's method? Read the theory first; it ends with the three bugs from last time.

Syntax you'll need

```
n = round(7.5 / 2.5)      # round() gives a whole number (int): 3
x = np.zeros(n + 1)       # array of n + 1 zeros
for k in range(n):        # k = 0, 1, ..., n - 1
    x[k + 1] = x[k] + 2   # fill the next slot from the current one
x[-1]                     # last element of an array
abs(a - b)                # absolute value
```

Tasks

1. **Euler step** in `euler_energy(dt, t_end)`. TODO 1a: the number of steps `n`. TODO 1b: one Euler step, $W_{k+1} = W_k + \Delta t\,(P - W_k/\tau_E)$.
2. **Error and ratio** in `main()`. TODO 2a: the absolute error at `t_end`, $|W_{\text{Euler}} - W_{\text{exact}}|$ (`exact_energy` is given). TODO 2b: the previous error divided by this one.

   Hint: `W` holds the whole time history; you only want its last value.
3. **Conclusion.** Look at the ratio column and complete the sentence in the comment at the bottom of the file.

Check yourself

For $\Delta t = 1$ s, Euler gives $W(4\,\text{s}) = 18.75$ MJ against the exact 17.29 MJ, so the error is about 1.457 MJ.

Submit

Commit and push `solution.py`. Optional: replace `??` on its first line with how long it took you (e.g. `# time: 20 min`).

THEORY

Euler's method, a refresher

A short recap of the first Euler note. In Tao Pang this is chapter 4 (ordinary differential equations), the first section on initial-value problems.

The method

For $\frac{dy}{dt} = f(t, y)$ with $y(0) = y_0$, Euler steps along the current slope:

$$y_{k+1} = y_k + \Delta t\, f(t_k, y_k)$$

Each step drops the $\frac{1}{2}\Delta t^2 y''$ term of the Taylor series, so its **local error** is $\sim \Delta t^2$. Reaching a fixed time T takes $T/\Delta t$ steps, so the **global error** is $\sim (T/\Delta t)\,\Delta t^2 \sim \Delta t$.

Reading the order from a table

If the global error behaves like $E \approx C\,\Delta t^p$, then halving Δt gives

$$\frac{E(\Delta t)}{E(\Delta t/2)} \approx \frac{C\,\Delta t^p}{C\,(\Delta t/2)^p} = 2^p$$

So a ratio near 2 means $p = 1$, near 4 means $p = 2$, near 16 means $p = 4$.

Worked example (different numbers, same steps)

A coil with $L = 4$ H and $R = 2\ \Omega$ is switched onto $V = 12$ V: $\frac{dI}{dt} = \frac{V - RI}{L}$, $I(0) = 0$. With $\Delta t = 0.5$ s:

$$I_1 = 0 + 0.5 \cdot \frac{12 - 2 \cdot 0}{4} = 1.5\ \text{A}, \qquad I_2 = 1.5 + 0.5 \cdot \frac{12 - 2 \cdot 1.5}{4} = 2.625\ \text{A}$$

The exact value is $I(1\,\text{s}) = 6\,(1 - e^{-0.5}) \approx 2.361$ A. Euler overshoots, because it uses the slope at the start of each step, and the slope keeps decreasing.

The three bugs from last time

- **Points vs steps.** n steps need $n + 1$ points. `np.linspace(0, t_end, n + 1)`, and its third argument must be an `int`, so compute `n = round(t_end / dt)` first.
- **Loop range.** If each pass writes `W[k + 1]`, the loop is `range(n)`. With `range(n + 1)` the last pass writes past the end (`IndexError`).
- **Array vs element.** `W` is the whole history; `W[-1]` is its last value. Subtracting a whole array where you meant one number gives `ValueError: setting an array element with a sequence`.